

Datastrukturer och Algoritmer

Författare - Lucas Berg

Inlämning

Uppgiften ska lämnas in som ett textdokument (doc, docx eller pdf) med max 2 sidor på Omniway.

I dokumentet ska det tydligt framkomma vilken uppgift det är, vilket svar som tillhör vilken fråga samt ditt namn.

Uppgiftens målsättning

Studenten ska kunna förklara grundläggande begrepp samt visa på en förståelse för när olika algoritmer och datastrukturer bör användas samt kunna resonera kring deras fördelar och nackdelar.

Uppgift

1: Algoritmer

En vektor ska sorteras: nämn tre olika sorteringsalgoritmer, ge exempel för varje exempel på ett fall då den är bättre än de två andra och motivera varför.

Quick Sort: Är bäst för medelstora osorterade samlingar när man vill ha snabbast genomsnittlig tid utan extra minnesallokering. Ett element (pivot) väljs, allt mindre flyttas till vänster och allt större till höger, sedan upprepas det rekursivt på båda delarna. Average och best case $O(n \log n)$, worst case $O(n^2)$.

Merge Sort: Är bra när man vill ha en pålitlig algoritm med samma komplexitet oavsett indata, alltid $O(n \log n)$. Samlingen delas på mitten tills bara enskilda element återstår, som sedan slås ihop parvis i sorterad ordning. Fördelen är den garanterade komplexiteten och att sorteringen är stabil.

Insertion Sort: Är bäst på liten eller nästan sorterad data, där den närmar sig $O(n)$. Varje element flyttas bakåt genom den redan sorterade delen tills det hamnar rätt. Quick och Merge Sort har overhead från rekursion och sammanslagning som gör dem långsammare på små samlingar, därför byter C++ `std::sort` till Insertion Sort för små partitioner.

2: Begrepp

Vilka olika typer av komplexitet är relevanta att mäta och varför är komplexitetsbegreppet användbart?

Time complexity, vilket är hur många operationer algoritmen gör i förhållande till mängden data.

Space complexity, vilket är hur mycket extra minne algoritmen behöver.

Båda mäts i tre fall: Best case som är bästa möjliga indata, average case som är typiskt data samt worst case som är värsta möjliga indata.

Det är användbart eftersom det möjliggör jämförelse av sorteringsalgoritmer utan att behöva bry sig om hur kraftfull datorn är. $O(\log n)$ slår alltid $O(n^2)$ oavsett om du kör på en snabb eller långsam maskin.

3: Datastrukturer

Givet en heap och en kö, ange för var datastruktur ett fall där den är bättre än den andra samt motivera.

Heap organiserar elementen beroende på storlek och föredras när man behöver plocka ut minsta/största element ur samlingen. Detta kan exempelvis vara ett typ av vapen manager system som alltid behöver komma åt vapnen med högst skada. Heaps time complexity är $O(\log n)$, medan en kö tvingas linjärsöka igenom hela samlingen i $O(n)$ för varje uttag.

Kö håller element beroende på när de läggs in/tas ut och föredras när man vill hålla koll på insättningsordningen av element i samlingen. Detta används i BFS då det ger enqueue/dequeue i $O(1)$ med minimal overhead.

4: A*

Vad har följande förkortningar för innebörd:

- G: Den faktiska, ackumulerade kostnaden från startnoden fram till den här noden längs den väg man hittills tagit dit.
- H: En uppskattning av kostnaden från den här noden till målnoden. Den beräknas, inte mäts, t.ex. med Manhattan- eller euklidiskt avstånd. För att A* ska garantera optimal väg måste heuristiken vara *admissible*, alltså aldrig överskatta den verkliga återstående kostnaden.
- F: Summan, $F = G + H$. Den totala uppskattade kostnaden för en väg som går genom den här noden. Det är F som algoritmen sorterar sin öppna lista på: noden med lägst F plockas närmast.

5: Pathfinding

Vad är skillnaden mellan Dijkstras och A* och i vilka situationer vill du använda den ena eller den andra?

Skillnaden är just heuristiken. Dijkstra expanderar noder enbart efter G, vilket är den faktiska kostnaden från start, och breder därför ut sig jämnt åt alla håll. A* expanderar efter $F = G + H$ och blir därmed riktad. Heuristiken drar sökningen mot målet och gör att långt färre noder besöks. Dijkstra är alltså A* med $H = 0$. Dijkstra föredras att användas när det inte finns ett enskilt mål, utan söker närmaste av flera. Även när du vill ha kortast väg från en nod till alla andra. Dessutom om man inte har någon vettig heuristik i exempelvis abstrakta grafer då det inte finns något att uppskatta med, så föredras Dijkstras. A* föredras när man har ett känt mål, en känd startpunkt, och en graf struktur som är inbäddad i rummet (all navigering på grid eller navmesh).

6: Grid

a) Ge två exempel på situationer där man **inte** vinner prestanda på att använda en Grid?

- 1) - När objekten är ojämnt fördelade. Ligger allt i samma cell gör du fortfarande n^2 tester där, samtidigt som du betalar för att bygga och underhålla gridet och för alla tomma celler.
- 2) - När objekten är mycket större än cellstorleken. Ett objekt som spänner över trettio celler måste testas i alla trettio, och duplicerade par måste filtreras bort.

b) Vad påverkar beslutet om storleken på noderna i en Grid och varför?

Storleken är en avvägning mellan precision och kostnad. Agentens storlek avgör vad en cell måste kunna representera, och världens finaste detaljer som smala passager och dörröppningar sätter taket för hur grovt du får gå. Neråt begränsas du av minne och söktid, eftersom halverad cellstorlek fyrdubblar antalet noder i 2D och därmed antalet noder A* måste expandera. Kör du path smoothing efteråt kan du komma undan med ett grovare grid.

7: QuadTree

Nämn två fördelar som ett QuadTree har jämfört med en Grid.

1) - Den anpassar sig efter datan. Trädet delar bara upp där det faktiskt finns objekt, så täta områden får fin uppdelning och tomma ytor kostar nästan ingenting. Ett grid betalar för alla celler oavsett om de används.

2) - Den skalar bättre med varierande objektstorlek och stora världar. Ett stort objekt kan ligga kvar på en högre nivå i trädet istället för att dupliceras över trettio celler, och minnet växer med antalet objekt istället för med världens yta.

8: QctTree

Vad skiljer ett Loose OctTree från ett vanligt OctTree och vad är fördelen?

I ett vanligt octree fastnar objekt som korsar en nodgräns i en föräldranod eller måste dupliceras. I ett loose octree utökas varje nods bounds med en marginal så noderna överlappar, vilket låter objekt sjunka ner till rätt nivå oavsett position. Det ger snabbare insättning, enklare uppdatering och jämnare fördelning.

9: QuadTree & QctTree

Vad innebär "djup" när man talar om QuadTrees och OctTrees.

Djupet är hur många gånger trädet har delat upp sig från rotnoden. Rotnoden är djup 0, dess barn djup 1, och så vidare. Ju djupare trädet går desto finare upplösning får du i det området, men också fler noder att hantera. Vanligtvis sätter man ett maxdjup för att undvika att trädet delar sig i oändlighet kring tätt packade objekt.

10: kD-Tree

a) Vad är fördelen med ett kD-tree mot ett QuadTree?

Ett kD-tree delar längs en axel i taget och kan välja delningsplan efter var datan faktiskt ligger, vilket ger tätare och mer balanserade träd. Det gör nearest-neighbour-sökningar snabbare, särskilt i högre dimensioner.

b) I vilka situationer gör sig ett QuadTree bättre än ett kD-tree?

Ett quadtree är bättre när objekt ofta rör sig, eftersom strukturen är rumsligt fix och enkel att uppdatera. Ett kD-tree bygger sin uppdelning utifrån datans fördelning och måste i princip byggas om när datan ändras, vilket gör det opraktiskt för dynamiska scener.